

Une feuille A4 de notes manuscrites autorisée

peut préciser tout un tas d'informations comme le numéro de la salle, le bâtiment, les caractéristiques spécifiques de la salle, ... S'il n'a pas trouvé de salle disponible pour cette demande, il envoie la chaîne de caractères « **NORESA** ». Lorsque le serveur a terminé d'envoyer son message, il ferme sa connexion en écriture.

- Étape IV Lorsqu'il reçoit la réponse du serveur, le client analyse la réponse. Si la réponse du serveur est négative, c'est-à-dire que la réservation n'est pas possible, alors le client termine et, ensuite, le serveur termine sa connexion avec le client. Sinon, le client répond au serveur, « **ROK** » s'il a reçu une proposition de réservation qui lui convient ou « **RNO** » s'il ne veut pas accepter la réservation proposée, puis il termine.
- Étape V Le serveur à la réception du message du client ôte la réservation s'il a reçu le message « **RNO** », puis il termine.

Dans cet exercice, vous ne gèrerez pas les erreurs de protocoles. Soit la structure suivante :

```
struct resa { uint8_t jour, heure, minute, duree, semaine, repetition, effectif, id; };
```

Dans les deux questions suivantes, vous écrirez une partie du code du client hybride **sur ipv4 et ipv6**.

1. Écrire le code du client qui réalise l'Étape I. (~15 lignes)
2. Écrire la fonction `void requete(int sock, struct resa r)` du client qui prend en paramètre la socket `sock` de communication avec le serveur et les données pour la demande de réservation contenues dans la structure `r` et réalise l'Étape II. (~10 lignes)

On suppose dans la suite de l'exercice que l'on dispose des fonctions suivantes :

- `int req-resa(struct resa r)` qui prend en paramètre les données pour la réservation d'une salle dans la structure `struct resa` et retourne le numéro d'une salle convenant à la demande s'il y en a une, et 0 sinon.
- `char * info-salle(int num)` qui prend en paramètre le numéro d'une salle et retourne une chaîne de caractères allouée dynamiquement et contenant les informations sur la salle si le numéro correspond bien à une salle, et `NULL` sinon.
- `int faire-resa(int num, struct resa r)` qui prend en paramètre le numéro d'une salle et les données pour la réservation d'une salle dans la structure `struct resa`. La fonction retourne le numéro de la salle si la réservation a pu être réalisée (modification du fichier de réservation des salles) et 0 sinon.
- `int oter-resa(int num, struct resa r)` qui prend en paramètre le numéro d'une salle et la réservation à annuler dans la structure `struct resa`. La fonction retourne 0 si la réservation a pu être annulée et 1 sinon.

Vous allez maintenant écrire le code du serveur sur `ipv6`. On veut que celui-ci puisse accepter des clients en parallèle. Il faudra pour cela **utiliser des threads** et les **variables globales sont interdites**.

On suppose qu'une seule instance du serveur tourne, ce qui signifie que seule cette instance peut consulter et modifier le fichier de réservation des salles.

3. Écrire le code de la fonction `struct resa req-client(int sock)` qui prend en paramètre une socket de communication avec un client et attend le message de requête de réservation d'une salle envoyé par le client, puis retourne une variable de type `struct resa` dont les champs sont remplis avec la requête du client. (~15 lignes)

4. Écrire le code de la fonction `void resa-client(int sock, ...)` qui prend en paramètre une socket de communication avec un client et d'autres arguments que vous devez déterminer (rappel : programme multi-threads) et réalise les Étape III à Étape V. (~25 lignes)
5. Écrire le code du serveur. (~25 lignes)

Exercice 2 On considère le code de l'application 1 suivant :

```

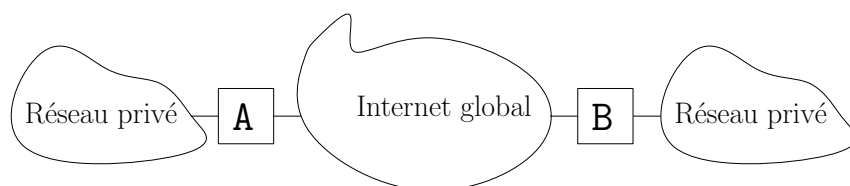
1 //Application 1
2
3 int main(int argc, char const *argv[]) {
4     int sock = socket(AF_INET6, SOCK_DGRAM, 0);
5     prepa_socket(sock, "ff12::55ba:cf94:f048:ab83", 2525);
6
7     int sock2 = socket(AF_INET6, SOCK_DGRAM, 0), sock3 = socket(AF_INET6, SOCK_DGRAM, 0);
8     prepa_socket(sock2, NULL, 1234);
9
10    struct sockaddr_in6 adr1 = prepa_adr(argv[1], 1234);
11    struct sockaddr_in6 adr2 = prepa_adr("ff12::c56:8a61:3cd5:45b1", 4322);
12    char buf[BUF_SIZE];
13
14    switch(fork()){
15    case 0:
16        while(1) {
17            memset(buf, 0, sizeof(buf));
18            recvfrom(sock, buf, sizeof(buf)-1, 0, NULL, NULL); //vient de l'application 2
19            sendto(sock2, buf, strlen(buf), 0, (struct sockaddr *) &adr1, sizeof(adr1)); //pour
20                                                //B
21        }
22    default:
23        close(sock);
24        while(1) {
25            memset(buf, 0, sizeof(buf));
26            recvfrom(sock2, buf, sizeof(buf)-1, 0, NULL, NULL); //vient de B
27            sendto(sock3, buf, strlen(buf), 0, (struct sockaddr *) &adr2, sizeof(adr2)); //pour
28                                                //l'application 2
29        }
30    }
31    return 0;
32 }
```

où :

- `void prepa_socket(int sock, const char *IP, int port)` prépare la socket `sock` pour recevoir des messages à l'adresse (`IP`, `port`). Si `IP` est `NULL`, alors la socket est préparée à recevoir sur n'importe quelle adresse,
- `struct sockaddr_in6 prepa_adr(const char *IP, int port)` retourne une adresse avec tous ses champs à 0 sauf les champs `sin6_addr` et `sin6_port` respectivement initialisés avec les arguments.

On rappelle qu'on ne se préoccupe pas ici des erreurs des appels systèmes. Vous considérerez de plus que les exécutions ne rencontrent pas de problème réseau.

On suppose que deux instances de l'application 1 tournent sur deux machines **A** et **B** n'appartenant pas à un même réseau privé et toutes les deux reliées à l'internet global comme montré sur la figure ci-après. Chacune de ces instances interagit également avec des instances d'une application 2 sur leur réseau privé.



1. Sur la machine B, la commande `ip` a donné la réponse suivante :

```
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc state UP group default qlen 1000
    link/ether 4a:49:43:49:79:bf brd ff:ff:ff:ff:ff:ff link-netnsid 0
    inet 192.23.37.206/24 brd 192.23.37.255 scope global eth0
        valid_lft forever preferred_lft forever
    inet6 fdc7::868:3d3b:b477:f7f1/64 scope global
        valid_lft forever preferred_lft forever
    inet6 fe80::ce81:b1c:bd2c:69e/64 scope link
        valid_lft forever preferred_lft forever
3: eth1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc state UP group default qlen 1000
    link/ether d0:94:66:06:d9:8e brd ff:ff:ff:ff:ff:ff
    inet 194.254.199.32/24 brd 194.254.199.255 scope global eth1
        valid_lft forever preferred_lft forever
    inet6 2001:660:3301:8070::32/64 scope global
        valid_lft forever preferred_lft forever
    inet6 fe80::d294:66ff:fe06:d98e/64 scope link
        valid_lft forever preferred_lft forever
4: eth2: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc state UP group default qlen 1000
    link/ether d0:94:66:06:d9:8e brd ff:ff:ff:ff:ff:ff permaddr d0:94:66:06:d9:8f
```

Quelle est la valeur de `argv[1]` pour l'instance de l'application 1 qui tourne sur la machine A si on veut qu'elle communique avec l'instance sur B ?

2. Décrire textuellement, étape par étape, le protocole de l'application 1.
3. Donner une valeur qui vous semble pertinente pour `BUF_SIZE` en expliquant votre choix.
4. Écrire le code de la fonction `prepa_socket`. (~10 lignes)
5. Écrire le code de la fonction `prepa_adr`. (~5 lignes)
6. on souhaite sécuriser la communication entre les machines A et B. Écrire textuellement les étapes qu'il faudrait coder pour permettre cette sécurisation.
7. En plus de communiquer avec l'instance sur la machine B, l'instance de l'application 1 sur la machine A communique, sur son réseau privé, avec des instances d'une application 2.

Écrire le code de l'application 2 qui doit en parallèle :

- recevoir les messages de l'application 1 et les afficher sur la sortie standard,
- lire sur l'entrée standard les messages de l'utilisateur et
 - envoyer chaque message à l'application 1 sauf si le message est égal à `safexit\n`,
 - si le message est égal à `safexit\n`, le programme termine dès qu'il ne reçoit plus aucun nouveau message ou qu'il ne lit plus rien sur l'entrée standard pendant 10s. Cela signifie que si un message est lu (respectivement reçu) avant la fin des 10s, celui-ci est envoyé à l'application 1 (respectivement affiché). Le programme termine seulement après que 10s se soient écoulées sans que rien ne soit lu ou reçu.

Vous déduirez le format des envois et réceptions de l'application 2, de ceux de l'application 1.

Pour cette question, vous devez **ne pas utiliser des processus système ou légers**. Vous pouvez utiliser les fonctions écrites pour l'application 1, dans le code de l'application 2. Pour lire une ligne (avec l'« `\n` » final inclus) sur l'entrée standard, vous pouvez utiliser la fonction `char * fgets(char * restrict str, int size, FILE * restrict stream)`. (~25 lignes)

Structures et prototypes de fonctions pouvant être utiles

```
int accept(int sockfd, struct sockaddr *addr, socklen_t *addrlen);
int bind(int sockfd, const struct sockaddr *addr, socklen_t addrlen);
int close(int fd);
int connect(int sockfd, const struct sockaddr *addr, socklen_t
addrlen);
int fcntl(int fd, int cmd, ... /* arg */ );
void FD_CLR(int fd, fd_set *set);
int  FD_ISSET(int fd, fd_set *set);
void FD_SET(int fd, fd_set *set);
void FD_ZERO(fd_set *set);
pid_t fork(void);
void freeaddrinfo(struct addrinfo *res);
int getaddrinfo(const char *node, const char *service,
                const struct addrinfo *hints, struct addrinfo **res);
uint32_t htonl(uint32_t hostlong);
uint16_t htons(uint16_t hostshort);
const char * inet_ntop(int af, const void * restrict src,
                      char * restrict dst, socklen_t size);
int inet_pton(int af, const char * restrict src, void * restrict dst);
int listen(int sockfd, int backlog);
void *memchr(const void s[.n], int c, size_t n);
int memcmp(const void *s1, const void *s2, size_t n);
void *memcpy(void *dest, const void *src, size_t n);
void *memmove(void *dest, const void *src, size_t n);
void *memset(void *s, int c, size_t n);
uint32_t ntohl(uint32_t netlong);
uint16_t ntohs(uint16_t netshort);
int open(const char *pathname, int flags, mode_t mode);
int poll(struct pollfd *fds, nfds_t nfds, int timeout);
int pthread_create(pthread_t *thread, const pthread_attr_t *attr,
                  void *(*start_routine) (void *), void *arg);
int pthread_mutex_destroy(pthread_mutex_t *mutex);
void pthread_exit(void *value_ptr);
int pthread_mutex_init(pthread_mutex_t *restrict mutex,
                      const pthread_mutexattr_t *restrict attr);
int pthread_join(pthread_t thread, void **retval);
int pthread_mutex_lock(pthread_mutex_t *mutex);
int pthread_mutex_unlock(pthread_mutex_t *mutex);
const char *inet_ntop(int af, const void *src, char *dst, socklen_t size);
int inet_pton(int af, const char *src, void *dst);
int poll(struct pollfd *fds, nfds_t nfds, int timeout);
ssize_t read(int fd, void *buf, size_t count);
ssize_t recv(int sockfd, void *buf, size_t len, int flags);
ssize_t recvfrom(int sockfd, void *buf, size_t len, int flags,
                struct sockaddr *src_addr, socklen_t *addrlen);
int select(int nfds, fd_set *readfds, fd_set *writefds,
```

```

        fd_set *exceptfds, struct timeval *timeout);
ssize_t send(int sockfd, const void *buf, size_t len, int flags);
ssize_t sendto(int sockfd, const void *buf, size_t len, int flags,
               const struct sockaddr *dest_addr, socklen_t addrlen);
int setsockopt(int sockfd, int level, int optname,
               const void *optval, socklen_t optlen);
int shutdown(int socket, int how);
int snprintf(char *str, size_t size, const char *format, ...);
int socket(int domain, int type, int protocol);
int sprintf(char *str, const char *format, ...);
int strcmp(const char *s1, const char *s2);
int strncmp(const char *s1, const char *s2, size_t n);
char *strcpy(char *dest, const char *src);
char *strncpy(char *dest, const char *src, size_t n);
pid_t waitpid(pid_t pid, int *wstatus, int options);
ssize_t write(int fd, const void *buf, size_t count);

struct sockaddr_in {
    short sin_family;
    unsigned short sin_port;
    struct in_addr sin_addr;
    char sin_zero[8];
}

struct sockaddr_in6 {
    u_int16_t sin6_family;
    u_int16_t sin6_port;
    u_int32_t sin6_flowinfo;
    struct in6_addr sin6_addr;
    u_int32_t sin6_scope_id;
}

struct pollfd {
    int fd;
    short events;
    short revents;
}

struct in_addr {
    unsigned long s_addr;
}

struct in6_addr {
    unsigned char s6_addr[16];
}

struct addrinfo {
    int ai_flags;
    int ai_family;
    int ai_socktype;
    int ai_protocol;
    socklen_t ai_addrlen;
    struct sockaddr *ai_addr;
    char *ai_canonname;
    struct addrinfo *ai_next;
}

struct ip_mreqn {
    struct in_addr imr_multiaddr;
    struct in_addr imr_address;
    int imr_ifindex;
};

struct ipv6_mreq {
    struct in6_addr ipv6mr_multiaddr;
    unsigned int ipv6mr_interface;
};

```